

```

In[1]:= ClearAll["Global`*"];
S14A[g_Graph] := Cases[EdgeList[g], _UndirectedEdge];
S14F[g_Graph] := Cases[EdgeList[g], _DirectedEdge];
S14PQ[p_List] :=
  Length[p] === 2 && MatchQ[p[[1]], _UndirectedEdge] && MatchQ[p[[2]], _UndirectedEdge] &&
  p[[1]] != p[[2]] && Length[Intersection[List @@ p[[1]], List @@ p[[2]]] === 1;
S14U[g_Graph, w_Integer : 50] :=
  Module[{a = Reverse @ S14A[g], n, used = {}, u = {}, e1, e2}, n = Length[a];
  If[n < 2, Return[{}]];
  Do[e1 = a[[i]];
  If[! MemberQ[used, e1], Do[e2 = a[[j]];
  If[! MemberQ[used, e2] &&
  Length[Intersection[List @@ e1, List @@ e2]] === 1, AppendTo[u, {e1, e2}];
  used = Join[used, {e1, e2}];
  Break[], {j, i + 1, Min[i + w, n]}], {i, 1, n}]; u];
S14Apply[g_Graph, u_List] :=
  Module[{all = EdgeList[g], used, iv, wc, nv = {}, na = {}, nf = {}, ev = {}, p, x, y, z, w, rem},
  If[u === {}, Return[<|"Graph" -> g, "Events" -> {}, "Changed" -> False|>]];
  used = Flatten[u, 1];
  If[! AllTrue[u, S14PQ] || ! AllTrue[used, MemberQ[S14A[g], #] &] || Length[DeleteDuplicates[
  used]] != Length[used], Return[<|"Graph" -> g, "Events" -> {}, "Changed" -> False|>]];
  iv = Cases[VertexList[g], _Integer];
  wc = If[iv === {}, 0, Max[iv]];
  Do[y = First @ Intersection[List @@ p[[1]], List @@ p[[2]]];
  x = First @ Complement[List @@ p[[1]], {y}];
  z = First @ Complement[List @@ p[[2]], {y}];
  w = ++wc;
  AppendTo[nv, w];
  na = Join[na, {UndirectedEdge[x, z], UndirectedEdge[x, w], UndirectedEdge[w, z]}];
  nf = Join[nf, {DirectedEdge[x, y], DirectedEdge[y, z]}];
  AppendTo[ev, <|"x" -> x, "y" -> y, "z" -> z, "w" -> w|>, {p, u}];
  rem = Complement[all, used]; <|"Graph" -> Graph[Union[Join[VertexList[g], nv]],
  Union[Join[rem, na, nf]], "Events" -> ev, "Changed" -> True|>];
S14Step[g_Graph] := S14Apply[g, S14U[g]];
S14Init[nv_Integer : 4, nc_Integer : 8] := Module[{v, a, c, k, am, at, b}, v = Range[nv];
  a = Range[nv + 1, 2 * nv];
  c = Range[2 * nv + 1, 2 * nv + nc];
  k = 1000000000;
  am = k + 1;
  at = k + 2;
  b = Union[Join[v, a, c, {k, am, at}], <|"Memory" -> Graph[b, {}],
  "Work" -> Graph[b, {}], "Values" -> v, "Actions" -> a, "Cues" -> c, "K" -> k,

```

```

    "AM" → am, "AT" → at, "Base" → b, "NCue" → nc, "WritesOK" → True |>];
S14SM[s_Association, g_Graph] := Join[s, <|"Memory" → g |>];
S14SW[s_Association, g_Graph] := Join[s, <|"Work" → g |>];
S14Next[g_Graph, x_] :=
  DeleteDuplicates@Cases[S14F[g], DirectedEdge[a_, b_] /; a === x ⇒ b];
S14Prev[g_Graph, x_] := DeleteDuplicates@Cases[S14F[g], DirectedEdge[a_, b_] /; b === x ⇒ a];
S14Index[g_Graph] := Module[{v = VertexList[g], p, in, out}, p = List @@@ S14F[g];
  in = If[p === {}, <| |>, GroupBy[p, Last → First]];
  out = If[p === {}, <| |>, GroupBy[p, First → Last]]; <|"V" → v, "In" → in, "Out" → out |>];
S14MemReset[s_Association, e_List] :=
  Module[{g = s["Memory"], v}, v = Union[Join[VertexList[g], s["Base"], Flatten[List @@@ e]]];
  S14SM[s, Graph[v, Union[Join[S14F[g], e]]]];
S14WriteTriple[s_Association, t_List] := Module[{h, r, e, ok},
  h = S14MemReset[s, {UndirectedEdge[t[[1]], t[[2]], UndirectedEdge[t[[2]], t[[3]]}];
  r = S14Step[h["Memory"]];
  e = If[TrueQ[r["Changed"]] && Length[r["Events"]] === 1, First[r["Events"]], <| |>];
  ok = AssociationQ[e] && Lookup[e, "x", None] === t[[1]] &&
    Lookup[e, "y", None] === t[[2]] && Lookup[e, "z", None] === t[[3]];
  If[ok, Join[S14SM[h, r["Graph"]], <|"WritesOK" → TrueQ[Lookup[s, "WritesOK", True]] |>],
  Join[h, <|"WritesOK" → False |>]];
S14WriteTriples[s_Association, t_List] := Fold[S14WriteTriple, s, t];
S14Replay[s_Association, p_Association] :=
  S14WriteTriples[S14WriteTriples[s, p["PathTriples"]], p["MarkTriples"]];
S14WorkReset[s_Association, e_List] :=
  Module[{g = s["Work"], v}, v = Union[Join[VertexList[g], s["Base"], Flatten[List @@@ e]]];
  S14SW[s, Graph[v, Union[Join[S14F[g], e]]]];
S14Probe[s_Association, c_Integer, n_Integer] := Module[{q, h, r, e, ok},
  q = Min[Join[VertexList[s["Memory"]], VertexList[s["Work"]], {c, n}]] - 1;
  h = S14WorkReset[s, {UndirectedEdge[c, q], UndirectedEdge[q, n]}];
  r = S14Step[h["Work"]];
  e = If[TrueQ[r["Changed"]] && Length[r["Events"]] === 1, First[r["Events"]], <| |>];
  ok = AssociationQ[e] && Lookup[e, "x", None] === c &&
    Lookup[e, "y", None] === q && Lookup[e, "z", None] === n; <|
  "State" → If[ok, S14SW[h, r["Graph"]], h], "OK" → ok, "Events" → If[ok, 1, 0] |>];
S14ActionRead[s_Association] := Module[{a, b}, a = S14Next[s["Work"], s["AT"]];
  If[! SameQ[a, {s["AM"]}], Return["Unknown"]];
  b = Intersection[S14Next[s["Work"], s["AM"]], s["Actions"]];
  Which[Length[b] === 1, First@FirstPosition[s["Actions"], First[b]],
  b === {}, "Unknown", True, "Ambiguous"];
S14Action[s_Association, j_Integer] := Module[{h, r, e, ok, t}, h = S14WorkReset[s,
  {UndirectedEdge[s["AT"], s["AM"]], UndirectedEdge[s["AM"], s["Actions"][[j]]}];
  r = S14Step[h["Work"]];

```

```

e = If[TrueQ[r["Changed"]] && Length[r["Events"]] === 1, First[r["Events"]], <| |>];
ok = AssociationQ[e] && Lookup[e, "x", None] === s["AT"] &&
  Lookup[e, "y", None] === s["AM"] && Lookup[e, "z", None] === s["Actions"][[j]];
t = If[ok, S14SW[h, r["Graph"], h]; <| "State" → t,
  "Executed" → If[ok, S14ActionRead[t, "Unknown"], "Events" → If[ok, 1, 0]|>];
S14Permute[s_Association, p_List] := Join[s, <| "Cues" → s["Cues"][[p]|>];
S14Relabel[s_Association, seed_Integer] :=
  Module[{v, nv, r, gm, gw}, v = Sort@Union[VertexList[s["Memory"]], VertexList[s["Work"]]];
  SeedRandom[seed];
  nv = Sort@RandomSample[Range[1000, 999 + 10 * Length[v]], Length[v]];
  r = Thread[v → nv];
  gm = Graph[VertexList[s["Memory"]] /. r, EdgeList[s["Memory"]] /. r];
  gw = Graph[VertexList[s["Work"]] /. r, EdgeList[s["Work"]] /. r];
  Join[s, <| "Memory" → gm, "Work" → gw, "Values" → s["Values"] /. r,
    "Actions" → s["Actions"] /. r, "Cues" → s["Cues"] /. r, "K" → s["K"] /. r,
    "AM" → s["AM"] /. r, "AT" → s["AT"] /. r, "Base" → s["Base"] /. r |>]];
S14Erase[s_Association] := S14SM[s, Graph[VertexList[s["Memory"]], {}];
S14Flip[s_Association] := S14SM[s, Graph[VertexList[s["Memory"]],
  DirectedEdge @@ Reverse[List @@ #] & /@ S14F[s["Memory"]]]];

S14TrainMarks[s_, c_, l_, "Chain", id_] :=
  Module[{q = s["Cues"][[l]], r, a, b}, r = -6 000 000 000 - 100 000 * id;
  {a, b} = r - Range[2];
  {{q, a, b}, {a, b, c}}];
S14TrainMarks[s_, c_, l_, "Parallel2", id_] :=
  Module[{q = s["Cues"][[l]], r, a, b, d, e, f, h}, r = -6 100 000 000 - 100 000 * id;
  {a, b, d, e, f, h} = r - Range[6];
  {{q, a, b}, {a, b, d}, {b, d, c}, {q, e, f}, {e, f, h}, {f, h, c}}];
S14TrainMarks[s_, c_, l_, "InternalMerge", id_] :=
  Module[{q = s["Cues"][[l]], r, a, b1, b2, m, d}, r = -6 200 000 000 - 100 000 * id;
  {a, b1, b2, m, d} = r - Range[5];
  {{q, a, b1}, {a, b2, m}, {b1, m, d}, {m, d, c}}];
S14TestMarks[s_, c_, l_, "DiamondMerge", id_] :=
  Module[{q = s["Cues"][[l]], r, a, b, m, d}, r = -6 300 000 000 - 100 000 * id;
  {a, b, m, d} = r - Range[4];
  {{q, a, m}, {q, b, m}, {m, d, c}}];
S14TestMarks[s_, c_, l_, "TwoStageMerge", id_] :=
  Module[{q = s["Cues"][[l]], r, a, b, m1, d, e, m2, f}, r = -6 400 000 000 - 100 000 * id;
  {a, b, m1, d, e, m2, f} = r - Range[7];
  {{q, a, m1}, {q, b, m1}, {m1, d, m2}, {m1, e, m2}, {m2, f, c}}];

S14BuildProgram[s_, mf_, top_, dep_, j_, q_, br_ : 3, seed_ : 1, tag_ : 0] := Module[

```

```

{main, good, a, pt = {}, mt = {}, cp = {}, i, b, cur, nxt, c, tg, l, id}, SeedRandom[seed];
main = Join[{s["K"]}, Table[-1500000000 - tag*10000000 - i*10000, {i, dep - 1}],
  {s["Values"][[j]]}];
good = RandomInteger[{1, br}, dep];
a = DeleteCases[Range[s["NCue"]], q];
Do[cur = main[[i]];
  nxt = main[[i + 1]];
  Do[c = -1600000000 - tag*10000000 - i*100000 - b*1000;
    tg = If[b === good[[i]], nxt, -1700000000 - tag*10000000 - i*100000 - b*1000];
    l = If[b === good[[i]], q, a[[1 + Mod[b - 1, Length[a]]]];
    id = tag*1000000 + i*100 + b;
    AppendTo[pt, {cur, c, tg}];
    mt = Join[mt, mf[s, c, l, top, id]];
    If[b === good[[i]], AppendTo[cp, {cur, c, tg}], {b, br}], {i, dep}]; <|"Topology" → top,
  "PathTriples" → pt, "MarkTriples" → mt, "CorrectPathTriples" → cp,
  "Depth" → dep, "Value" → j, "Cue" → q, "Branches" → br |>];
S14TrainProgram[s_, top_, d_, j_, q_, br_: 3, seed_: 1, tag_: 0] :=
  S14BuildProgram[s, S14TrainMarks, top, d, j, q, br, seed, tag];
S14TestProgram[s_, top_, d_, j_, q_, br_: 3, seed_: 1, tag_: 0] :=
  S14BuildProgram[s, S14TestMarks, top, d, j, q, br, seed, tag];

S14Transition[z_Integer, m_Integer, tr_List] :=
  If[m === 1, tr[[2]], If[z === 0, 0, Min[3, z + tr[[1]]]];
S14GateFeat[i_, v_, z_] :=
  N@{1, Length@Lookup[i["In"], v, {}], Length@Lookup[i["Out"], v, {}], Boole[z > 0], z};

S14PropI[i_Association, c_, tr_List, wg_List, max_Integer: 8] :=
  Module[{v = i["V"], h, z, raw, nz, h2, a = <| |>, new, same, r = 0, k, ps, pz, m, x},
    h = AssociationThread[v, ConstantArray[0, Length[v]]];
    z = AssociationThread[v, ConstantArray[0, Length[v]]];
    AssociateTo[h, c → 1];
    AssociateTo[a, c → 0];
    Do[
      raw = AssociationThread[v, Boole[Total@Lookup[h, Lookup[i["In"], #, {}], 0] > 0] & /@ v];
      nz = <| |>;
      Do[If[Lookup[raw, x, 0] === 0, AssociateTo[nz, x → 0],
        ps = Select[Lookup[i["In"], x, {}], Lookup[h, #, 0] === 1 &];
        pz = If[ps === {}, 0, Max@Lookup[z, ps, 0]];
        m = Boole[Length[ps] ≥ 2];
        AssociateTo[nz, x → S14Transition[pz, m, tr]], {x, v}];
      new = Select[v, Lookup[raw, #, 0] === 1 && !KeyExistsQ[a, #] &];
      Scan[(AssociateTo[a, # → k] &, new];

```

```

h2 = AssociationThread[v,
  Boole[Lookup[raw, #, 0] === 1 && wg.S14GateFeat[i, #, Lookup[nz, #, 0] > 0] & /@ v];
same = SameQ[h2, h] && SameQ[AssociationThread[v, Lookup[nz, #, 0] & /@ v], z];
h = h2;
z = AssociationThread[v, Lookup[nz, #, 0] & /@ v];
r = k;
If[same, Break[], {k, 1, max}]; <|"Arrival" → a, "LatentState" → z, "Rounds" → r|>;
S14Prop[g_Graph, c_, tr_List, wg_List, max_Integer: 8] :=
  S14PropI[S14Index[g], c, tr, wg, max];

S14DecisionI[s_Association, i_Association, tr_List, wg_List, q_Integer,
  bud_Integer: 8] := Catch[Module[{p, a, c = s["K"], vis = {}, nx, t, mn, z, n, k, v},
  p = S14PropI[i, s["Cues"]][q], tr, wg, bud];
  a = p["Arrival"];
  Do[If[MemberQ[s["Values"], c], v = First@FirstPosition[s["Values"], c];
    Throw[<|"Status" → "Found", "Value" → v|>]];
  If[MemberQ[vis, c], Throw[<|"Status" → "Loop", "Value" → "Loop"|>]];
  AppendTo[vis, c];
  nx = Lookup[i["Out"], c, {}];
  If[nx === {}, Throw[<|"Status" → "Unknown", "Value" → "Unknown"|>]];
  If[Length[nx] === 1, n = First[nx], t = Lookup[a, nx, Infinity];
  mn = Min[t];
  If[mn === Infinity, Throw[<|"Status" → "Unknown", "Value" → "Unknown"|>]];
  z = Pick[nx, t, mn];
  If[Length[z] > 1, Throw[<|"Status" → "Ambiguous", "Value" → "Ambiguous"|>]];
  n = First[z];
  c = n, {k, 1, 3000}]; <|"Status" → "Budget", "Value" → "Budget"|>]];

S14Case[type_, s_, q_, st_, v_] := <|
  "Type" → type, "State" → s, "Index" → S14Index[s["Memory"]],
  "Cue" → q, "ExpectedStatus" → st, "ExpectedValue" → v|>;

S14TrainCases[] := Module[{tops = {"Chain", "Parallel2", "InternalMerge"}, a, b},
  a = Flatten[Table[Module[{s = S14Init[4, 8], top = tops[[ti]],
    p, m, x, w, j, q, br, k, seed}, j = 1 + Mod[d + ti - 2, 4];
    q = 1 + Mod[d + 2 * ti - 3, 8];
    br = 2 + Mod[d + ti, 2];
    k = Max[1, Ceiling[d/2]];
    seed = 10 000 * ti + 100 * d;
    p = S14TrainProgram[s, top, d, j, q, br, seed];
    m = S14Replay[s, p];
    x = First@FirstPosition[p["PathTriples"], p["CorrectPathTriples"][[k]]];
    w = 1 + Mod[q, 8];

```

```

    {S14Case["Positive", m, q, "Found", j], S14Case["Broken", S14Replay[s, Join[
      p, <|"PathTriples" → Delete[p["PathTriples"], x]|>]], q, "Unknown", "Unknown"],
      S14Case["WrongCue", m, w, "Unknown", "Unknown"]]}, {ti, 1, 3}, {d, 1, 4}, 2];
b = Flatten[Table[Module[{s = S14Init[4, 8], top = tops[[ti], q, u, v, m],
  q = 1 + Mod[d + ti - 2, 8];
  u = S14TrainProgram[s, top, d, 1, q, 3, 20000*ti + d, 1];
  v = S14TrainProgram[s, top, d, 4, q, 3, 21000*ti + d, 2];
  m = Fold[S14Replay, s, {u, v}];
  S14Case["Conflict", m, q, "Ambiguous", "Ambiguous"]]}, {ti, 1, 3}, {d, {2, 4}}, 1];
Join[a, b];

S14TransitionCandidates[] := Flatten[Table[{a, b}, {a, {1, 2}}, {b, {1, 2, 3}}, 1];
S14GateCandidates[] := Tuples[{Range[1, 3], {-2, -1, 0}, {-1, 0, 1}, {0, 1, 2}, {-2, -1, 0}}];
S14CasePassQ[c_Association, tr_List, wg_List] :=
  Module[{r = S14DecisionI[c["State"], c["Index"], tr, wg, c["Cue"], 8]},
    If[c["ExpectedStatus"] === "Found", r["Status"] === "Found" &&
      r["Value"] === c["ExpectedValue"], r["Status"] === c["ExpectedStatus"]];

S14Fit[] :=
  Module[{c = S14TrainCases[], trs = S14TransitionCandidates[], wgs = S14GateCandidates[],
    alive, ord, best, p}, alive = Flatten[Table[{tr, wg}, {tr, trs}, {wg, wgs}], 1];
  ord = SortBy[c, Switch[#, "Type", "Broken", 1, "Conflict",
    2, "WrongCue", 3, "Positive", 4, _, 5] &];
  Do[alive = Select[alive, S14CasePassQ[x, #1, #2] &];
  If[alive === {}, Break[], {x, ord}];
  If[alive === {}, Return[<|"ActWeights" → {0, 0, 1, 0}, "SearchSpace" →
    Length[trs]*Length[wgs], "TrainingCases" → Length[c], "Perfect" → False,
    "SurvivingPolicies" → 0, "IntermediateLabels" → 0, "StopLabels" → 0,
    "ExplicitCandidates" → False, "Objective" → "FinalOutcomeOnly"|>]];
  best = First@SortBy[alive, {Total@Abs@Flatten[#] &, Count[Flatten[#], Except[0]] &}];
  p = Select[c, S14CasePassQ[#, best[[1]], best[[2]]] &]; <|"ActWeights" → {0, 0, 1, 0},
    "TransitionParams" → best[[1]], "GateWeights" → best[[2]], "LatentStates" → 4,
    "SearchSpace" → Length[trs]*Length[wgs], "TrainingCases" → Length[c],
    "Reward" → Length[p], "TrainingAccuracy" → N[Length[p]/Length[c]],
    "Perfect" → Length[p] === Length[c], "SurvivingPolicies" → Length[alive],
    "IntermediateLabels" → 0, "StopLabels" → 0, "ExplicitCandidates" → False,
    "Objective" → "FinalOutcomeOnly", "PassedTypes" → Counts[Lookup[p, "Type"]]|>];
S14FitTest[p_Association] := <|"ActWeights" → Lookup[p, "ActWeights"],
  "TransitionParams" → Lookup[p, "TransitionParams", "-"],
  "GateWeights" → Lookup[p, "GateWeights", "-"], "LatentStates" →
  Lookup[p, "LatentStates", 4], "SearchSpace" → Lookup[p, "SearchSpace"],
  "TrainingCases" → Lookup[p, "TrainingCases"], "Reward" → Lookup[p, "Reward", "-"],

```

```

"TrainingAccuracy" → Lookup[p, "TrainingAccuracy", "-"],
"IntermediateLabels" → Lookup[p, "IntermediateLabels", 0],
"StopLabels" → Lookup[p, "StopLabels", 0],
"ExplicitCandidates" → Lookup[p, "ExplicitCandidates", False], "Perfect" →
  Lookup[p, "Perfect", False], "AllPassed" → TrueQ[Lookup[p, "Perfect", False] &&
    Lookup[p, "TrainingAccuracy", 0] === 1. && Lookup[p, "IntermediateLabels", 1] === 0 &&
    Lookup[p, "StopLabels", 1] === 0 && !Lookup[p, "ExplicitCandidates", True]]];
S14Read[s_Association, p_Association, q_Integer] :=
  Catch[Module[{t, mem, pr, a, c, vis = {}, path, ev = 0, nx, z, mn, g, n, h, k, v},
    t = S14SW[s, Graph[s["Base"], {}]];
    mem = EdgeList[t["Memory"]];
    pr = S14Prop[t["Memory"], t["Cues"][[q]], p["TransitionParams"], p["GateWeights"], 8];
    a = pr["Arrival"];
    c = t["K"];
    path = {c};
    Do[If[MemberQ[t["Values"], c], v = First@FirstPosition[t["Values"], c];
      Throw[<| "State" → t, "Status" → "Found", "Value" → v, "Hops" → Length[path] - 1,
        "ProbeEvents" → ev, "MemoryUnchanged" → EdgeList[t["Memory"]] === mem |>]];
    If[MemberQ[vis, c], Throw[<| "State" → t, "Status" → "Loop", "Value" → "Loop",
      "Hops" → Length[path] - 1, "ProbeEvents" → ev, "MemoryUnchanged" → True |>]];
    AppendTo[vis, c];
    nx = S14Next[t["Memory"], c];
    If[nx === {}, Throw[<| "State" → t, "Status" → "Unknown", "Value" → "Unknown",
      "Hops" → Length[path] - 1, "ProbeEvents" → ev, "MemoryUnchanged" → True |>]];
    If[Length[nx] === 1, n = First[nx], z = Lookup[a, nx, Infinity];
      mn = Min[z];
      If[mn === Infinity, Throw[<| "State" → t, "Status" → "Unknown", "Value" → "Unknown",
        "Hops" → Length[path] - 1, "ProbeEvents" → ev, "MemoryUnchanged" → True |>]];
      g = Pick[nx, z, mn];
      If[Length[g] > 1,
        Throw[<| "State" → t, "Status" → "Ambiguous", "Value" → "Ambiguous",
          "Hops" → Length[path] - 1, "ProbeEvents" → ev, "MemoryUnchanged" → True |>]];
      n = First[g];
      h = S14Probe[t, c, n];
      If[! TrueQ[h["OK"]],
        Throw[<| "State" → t, "Status" → "ProbeError", "Value" → "ProbeError",
          "Hops" → Length[path] - 1, "ProbeEvents" → ev, "MemoryUnchanged" → True |>]];
      t = h["State"];
      ev += h["Events"];
      c = n;
      AppendTo[path, c], {k, 1, 3000}]]];
S14Execute[s_Association, p_Association, q_Integer] := Module[{r, a}, r = S14Read[s, p, q];

```

```

If[r["Status"] != "Found", Return[Join[r, <| "Selected" → "NONE",
  "Executed" → "Unknown", "TotalEvents" → r["ProbeEvents"] |>]]];
a = S14Action[r["State"], r["Value"]];
Join[r, <| "Selected" → r["Value"], "Executed" → a["Executed"],
  "TotalEvents" → r["ProbeEvents"] + a["Events"], "MemoryUnchanged" → And[
    r["MemoryUnchanged"], EdgeList[a["State"]["Memory"]] === EdgeList[s["Memory"]] |>]];

S14One[p_, top_, d_, j_, q_, br_ : 3, seed_ : 1, pm_ : True, rl_ : True] :=
Module[{s = S14Init[4, 8], x, pr, m, e}, If[pm, SeedRandom[seed + 999];
  x = RandomSample[Range[8]];
  s = S14Permute[s, x]];
pr = S14TestProgram[s, top, d, j, q, br, seed];
m = S14Replay[s, pr];
If[rl, m = S14Relabel[m, seed + 5000]];
e = S14Execute[m, p, q]; <| "Topology" → top, "Depth" → d, "Value" → j, "Cue" → q,
  "Status" → e["Status"], "Recall" → e["Value"], "Executed" → e["Executed"],
  "Hops" → e["Hops"], "Passed" → And[m["WritesOK"], e["Status"] === "Found",
    e["Value"] === j, e["Executed"] === j, e["Hops"] === 2 * d, e["MemoryUnchanged"]] |>];

S14DepthTest[p_, top_, ds_] := Flatten[
  Table[S14One[p, top, d, j, 1 + Mod[d + j - 2, 8], 3, 300 000 + 100 * d + j], {d, ds}, {j, 4}], 1];

S14Trial[p_, top_, seed_, max_ : 20] := Module[{d, j, q, b}, SeedRandom[seed];
  d = RandomInteger[{5, max}];
  j = RandomInteger[{1, 4}];
  q = RandomInteger[{1, 8}];
  b = RandomInteger[{2, 4}];
  Join[<| "Seed" → seed |>, S14One[p, top, d, j, q, b, seed]]];

S14Trials[p_, top_, n_ : 30, max_ : 20, seed_ : 20 260 807] :=
Table[S14Trial[p, top, seed + i, max], {i, n}];

S14BrokenTest[p_, top_, d_ : 8, j_ : 2, q_ : 2, k_ : 4] :=
Module[{s = S14Init[4, 8], pr, x, m, e}, pr = S14TestProgram[s, top, d, j, q, 3, 400 001];
  x = First@FirstPosition[pr["PathTriples"], pr["CorrectPathTriples"][[k]]];
  m = S14Replay[s, Join[pr, <| "PathTriples" → Delete[pr["PathTriples"], x] |>]];
  e = S14Execute[m, p, q]; <| "Test" → "Broken", "Topology" → top, "Status" → e["Status"],
    "Passed" → And[e["Status"] === "Unknown", e["Selected"] === "NONE"] |>];

S14WrongTest[p_, top_, d_ : 8, j_ : 2, q_ : 2] :=
Module[{s = S14Init[4, 8], pr, m, e, w}, pr = S14TestProgram[s, top, d, j, q, 3, 400 002];
  m = S14Replay[s, pr];
  w = 1 + Mod[q, 8];
  e = S14Execute[m, p, w]; <| "Test" → "WrongCue", "Topology" → top, "Status" →
    e["Status"], "Passed" → And[e["Status"] != "Found", e["Selected"] === "NONE"] |>];

S14ConflictTest[p_, top_, d_ : 8, q_ : 2] :=
Module[{s = S14Init[4, 8], a, b, m, e}, a = S14TestProgram[s, top, d, 1, q, 3, 400 003, 1];

```

```

b = S14TestProgram[s, top, d, 4, q, 3, 400 004, 2];
m = Fold[S14Replay, s, {a, b}];
e = S14Execute[m, p, q]; <| "Test" → "Conflict", "Topology" → top, "Status" →
  e["Status"], "Passed" → And[e["Status"] === "Ambiguous", e["Selected"] === "NONE"] |>;
S14Noise[s_, d_, tag_] :=
Module[{r = -9 000 000 000 - tag * 1 000 000, x, t}, x = Table[r - 10 000 * i, {i, 0, d}];
t = {#[1], #[1] - 1, #[2]} & /@ Partition[x, 2, 1]; <|
  "PathTriples" → t, "MarkTriples" → {} |>;
S14NoiseTest[p_, top_, d_ : 8, j_ : 2, q_ : 2, k_ : 10] :=
Module[{s = S14Init[4, 8], pr, no, m, e}, SeedRandom[20 260 807];
pr = S14TestProgram[s, top, d, j, q, 3, 400 005];
no = Table[S14Noise[s, RandomInteger[{1, d}], i], {i, k}];
m = Fold[S14Replay, s, Join[{pr}, no]];
e = S14Execute[m, p, q]; <| "Test" → "Noise",
  "Topology" → top, "Status" → e["Status"], "Recall" → e["Value"],
  "Passed" → And[e["Status"] === "Found", e["Value"] === j, e["Executed"] === j] |>;
S14CompressionTest[p_, top_, d_ : 8, j_ : 3, q_ : 2, r_ : 20] :=
Module[{s = S14Init[4, 8], pr, m0, m, e, g, u, c},
pr = S14TestProgram[s, top, d, j, q, 3, 400 006];
m0 = S14Replay[s, pr];
u = Length@S14F[m0["Memory"]];
m = Nest[S14Replay[#, pr] &, s, r];
e = S14Execute[m, p, q];
g = 2 * (Length[pr["PathTriples"]] + Length[pr["MarkTriples"]]) * r;
c = N[1 - Length@S14F[m["Memory"]]/g]; <| "Test" → "Compression",
  "Topology" → top, "Generated" → g, "Unique" → Length@S14F[m["Memory"],
  "ExpectedUnique" → u, "CompressionRate" → c, "Passed" →
  And[Length@S14F[m["Memory"]] === u, e["Value"] === j, e["Executed"] === j] |>;
S14HistoryTest[p_, top_, d_ : 6, q_ : 2] :=
Module[{s = S14Init[4, 8], a, b, m1, m2, e1, e2, x1, x2, f1, f2, z},
a = S14TestProgram[s, top, d, 1, q, 3, 500 001];
b = S14TestProgram[s, top, d, 3, q, 3, 500 001];
m1 = S14Replay[s, a];
m2 = S14Replay[s, b];
e1 = S14Execute[m1, p, q];
e2 = S14Execute[m2, p, q];
x1 = S14Execute[S14Erase[m1], p, q];
x2 = S14Execute[S14Erase[m2], p, q];
f1 = S14Execute[S14Flip[m1], p, q];
f2 = S14Execute[S14Flip[m2], p, q];
z = {EdgeList[m1["Work"]] === EdgeList[m2["Work"]],
  EdgeList[m1["Memory"]] != EdgeList[m2["Memory"]], e1["Value"] === 1, e2["Value"] === 3,
  x1["Status"] === "Unknown", x2["Status"] === "Unknown", f1["Status"] === "Unknown",

```

```
f2["Status"] === "Unknown"}; <| "Test" → "History", "Topology" → top,
"RecallA" → e1["Value"], "RecallB" → e2["Value"], "AllPassed" → And @@ z |>];
```

```
S14CoreAudit[] := Module[{e12 = UndirectedEdge[1, 2],
  e23 = UndirectedEdge[2, 3], g, a, b, f1, f2}, g = Graph[{1, 2, 3}, {e12, e23}];
a = S14Apply[g, {{e12, e23}}];
b = S14Apply[g, {{e23, e12}}];
f1 = S14F[a["Graph"]];
f2 = S14F[b["Graph"]]; <|
  "ForwardFrozen" → And @@ (MemberQ[f1, #] & /@ {DirectedEdge[1, 2], DirectedEdge[2, 3]}),
  "ReverseFrozen" → And @@ (MemberQ[f2, #] & /@ {DirectedEdge[3, 2], DirectedEdge[2, 1]}),
  "OrderSensitive" → f1 != f2, "ThreeActiveEdges" → Length[S14A[a["Graph"]]] === 3,
  "AllPassed" → And[And @@ (MemberQ[f1, #] & /@ {DirectedEdge[1, 2], DirectedEdge[2, 3]}),
    And @@ (MemberQ[f2, #] & /@ {DirectedEdge[3, 2], DirectedEdge[2, 1]}),
    f1 != f2, Length[S14A[a["Graph"]]] === 3] |>];
```

```
S14Audit[] := Module[{d, g, c, n, t}, d = {DownValues[S14TrainCases],
  DownValues[S14Transition], DownValues[S14PropI], DownValues[S14Fit]};
g = FreeQ[d, GraphDistance | FindShortestPath | ShortestPathFunction];
c = FreeQ[d, "Candidates"];
n = FreeQ[d, "SeenMerge" | "PostMergeAge"];
t = FreeQ[DownValues[S14TrainCases], "DiamondMerge" | "TwoStageMerge"]; <|
  "ActWeightsFixed" → {0, 0, 1, 0}, "LatentStates" → 4,
  "NamedProvenanceChannelsAbsent" → n, "TransitionParamsSearched" → 2,
  "IntermediateLabels" → 0, "StopLabels" → 0, "ExplicitCandidatesAbsent" → c,
  "GraphDistanceAbsent" → g, "HeldOutTopologiesAbsentFromTraining" → t,
  "TrainingTopologies" → {"Chain", "Parallel2", "InternalMerge"},
  "HeldOutTopologies" → {"DiamondMerge", "TwoStageMerge"},
  "AllPassed" → And[g, c, n, t] |>];
```

```
S14TopologyAudit[] := Module[{s = S14Init[], a, b, ma, mb, ia, ib},
  a = S14TestProgram[s, "DiamondMerge", 1, 1, 1, 1, 1];
  b = S14TestProgram[s, "TwoStageMerge", 1, 1, 1, 1, 1];
  ma = S14Replay[s, a]["Memory"];
  mb = S14Replay[s, b]["Memory"];
  ia = Count[VertexList[ma], v_ /; Length[S14Prev[ma, v]] ≥ 2];
  ib = Count[VertexList[mb], v_ /; Length[S14Prev[mb, v]] ≥ 2]; <|
    "DiamondMergeNodes" → ia, "TwoStageMergeNodes" → ib,
    "DifferentMergeCounts" → ia != ib, "AllPassed" → And[ia ≥ 1, ib ≥ 2, ia != ib] |>];
```

```
In[62]:= Dataset[{S14CoreAudit[]}]
Dataset[{S14Audit[]}]
Dataset[{S14TopologyAudit[]}]
```

```
tc = S14TrainCases[];
Length[tc]
Counts[Lookup[tc, "Type"]]
```

Out[62]=

ForwardFrozen	True
ReverseFrozen	True
OrderSensitive	True
ThreeActiveEdges	True
AllPassed	True

Out[63]=

ActWeightsFixed	LatentStates	NamedProvenanceChannelsAbsent	TransitionParamsSearched	IntermediateL
{0, 0, 1, 0}	4	True	2	0

◀ columns 1–10 of 12 ▶

Out[64]=

DiamondMergeNodes	TwoStageMergeNodes	DifferentMergeCounts	AllPassed
2	3	True	True

Out[66]=

42

Out[67]=

◀|Positive → 12, Broken → 12, WrongCue → 12, Conflict → 6|▶

```
In[68]:= ClearAll[S14FitSlice];
```

```
S14FitSlice[tr_List] :=
  Module[{c = S14TrainCases[], g = S14GateCandidates[], ord, alive, best, p},
    ord = Join[c[{{11, 35}}], c[Complement[Range[Length[c]], {11, 35}]]];
    alive = g;
    Do[
      alive = Select[alive, S14CasePassQ[x, tr, #] &];
      If[alive === {}, Break[]],
      {x, ord}
    ];
    If[alive === {},
      Return[<|"ActWeights" → {0, 0, 1, 0}, "TransitionParams" → tr,
        "GateWeights" → "-", "LatentStates" → 4, "SliceSearchSpace" → Length[g],
        "SearchSpace" → 6 Length[g], "TrainingCases" → Length[c],
        "Reward" → "-", "TrainingAccuracy" → "-", "Perfect" → False,
        "SurvivingPolicies" → 0, "IntermediateLabels" → 0, "StopLabels" → 0,
        "ExplicitCandidates" → False, "Objective" → "FinalOutcomeOnly" |>]
    ];
    best = First@SortBy[alive, {Total@Abs[#] &, Count[#, Except[0]] &}];
    p = Select[c, S14CasePassQ[#, tr, best] &];
    <|"ActWeights" → {0, 0, 1, 0}, "TransitionParams" → tr,
      "GateWeights" → best, "LatentStates" → 4, "SliceSearchSpace" → Length[g],
      "SearchSpace" → 6 Length[g], "TrainingCases" → Length[c],
      "Reward" → Length[p], "TrainingAccuracy" → N[Length[p]/Length[c]],
      "Perfect" → Length[p] === Length[c], "SurvivingPolicies" → Length[alive],
      "IntermediateLabels" → 0, "StopLabels" → 0, "ExplicitCandidates" → False,
      "Objective" → "FinalOutcomeOnly", "PassedTypes" → Counts[Lookup[p, "Type"]] |>
    ];
```

```
In[70]:= q1 = S14FitSlice[{1, 1}];  
Dataset[{q1}]
```

Out[71]=

ActWeights	TransitionParams	GateWeights	LatentStates	SliceSearchSpace	SearchSpace	TrainingCases	Re
{0, 0, 1, 0}	{1, 1}	2	4	243	1458	42	42
		-1					
		0					
		2					
		-1					

```
In[72]:= q2 = S14FitSlice[{1, 2}];  
Dataset[{q2}]
```

Out[73]=

ActWeights	TransitionParams	GateWeights	LatentStates	SliceSearchSpace	SearchSpace	TrainingCases	Re
{0, 0, 1, 0}	{1, 2}	–	4	243	1458	42	–

```
In[74]:= q3 = S14FitSlice[{1, 3}];  
Dataset[{q3}]
```

Out[75]=

ActWeights	TransitionParams	GateWeights	LatentStates	SliceSearchSpace	SearchSpace	TrainingCases	Re
{0, 0, 1, 0}	{1, 3}	–	4	243	1458	42	–

```
In[76]:= q4 = S14FitSlice[{2, 1}];  
Dataset[{q4}]
```

Out[77]=

ActWeights	TransitionParams	GateWeights	LatentStates	SliceSearchSpace	SearchSpace	TrainingCases	Re
{0, 0, 1, 0}	{2, 1}	3	4	243	1458	42	42
		-2					
		1					
		2					
		-1					

```
In[78]:= q5 = S14FitSlice[{2, 2}];
Dataset[{q5}]
```

```
Out[79]=
```



ActWeights	TransitionParams	GateWeights	LatentStates	SliceSearchSpace	SearchSpace	TrainingCases	Re
{0, 0, 1, 0}	{2, 2}	–	4	243	1458	42	–

K < columns 1–10 of 15 > X

```
In[80]:= q6 = S14FitSlice[{2, 3}];
Dataset[{q6}]
```

```
Out[81]=
```



ActWeights	TransitionParams	GateWeights	LatentStates	SliceSearchSpace	SearchSpace	TrainingCases	Re
{0, 0, 1, 0}	{2, 3}	–	4	243	1458	42	–

K < columns 1–10 of 15 > X

```
In[82]:= qs = {q1, q2, q3, q4, q5, q6};
```

```
Dataset[
  KeyTake[#,
    {"TransitionParams", "GateWeights", "TrainingCases",
      "Reward", "TrainingAccuracy", "Perfect", "SurvivingPolicies"}
  ] & /@ qs
]
```

```
Out[83]=
```

TransitionParams	GateWeights	TrainingCases	Reward	TrainingAccuracy	Perfect	SurvivingPolicies
{1, 1}	{2, -1, 0, 2, -1}	42	42	1.0	True	3
{1, 2}	–	42	–	–	False	0
{1, 3}	–	42	–	–	False	0
{2, 1}	{3, -2, 1, 2, -1}	42	42	1.0	True	1
{2, 2}	–	42	–	–	False	0
{2, 3}	–	42	–	–	False	0

```
In[86]:= good = Select[qs, TrueQ[Lookup[#, "Perfect", False]] &];
Length[good]
```

```
Out[87]=
```

2

```
In[88]:= s14 = First@SortBy[
  good,
  Total@Abs@Join[Lookup[#, "TransitionParams"], Lookup[#, "GateWeights"]] &
];
```

Dataset[{{S14FitTest[s14]}}

Out[89]=

ActWeights	TransitionParams	GateWeights	LatentStates	SearchSpace	TrainingCases	Reward	TrainingAcc
{0, 0, 1, 0}	{1, 1}	2	4	1458	42	42	1.0
		-1					
		0					
		2					
		-1					

K < columns 1-10 of 13 > X

```
In[90]:= d1 = S14DepthTest[s14, "DiamondMerge", Range[5, 12]];
d2 = S14DepthTest[s14, "TwoStageMerge", Range[5, 12]];
```

```
Dataset[{
  <|"Topology" -> "DiamondMerge", "Cases" -> Length[d1], "Passed" ->
    Count[Lookup[d1, "Passed"], True], "AllPassed" -> And @@ Lookup[d1, "Passed"]|>,
  <|"Topology" -> "TwoStageMerge", "Cases" -> Length[d2], "Passed" ->
    Count[Lookup[d2, "Passed"], True], "AllPassed" -> And @@ Lookup[d2, "Passed"]|>
}]
```

Out[92]=

Topology	Cases	Passed	AllPassed
DiamondMerge	32	32	True
TwoStageMerge	32	32	True

```
In[93]:= Dataset[{
  S14HistoryTest[s14, "DiamondMerge"],
  S14BrokenTest[s14, "DiamondMerge"],
  S14WrongTest[s14, "DiamondMerge"],
  S14ConflictTest[s14, "DiamondMerge"],
  S14NoiseTest[s14, "DiamondMerge"],
  S14CompressionTest[s14, "DiamondMerge"]
}]
```

Out[93]=

△

Test	History
Topology	DiamondMerge
RecallA	1
RecallB	3
5 total	
Test	Broken
Topology	DiamondMerge
Status	Unknown
Passed	True
Test	WrongCue
Topology	DiamondMerge
Status	Unknown
Passed	True
Test	Conflict
Topology	DiamondMerge
Status	Ambiguous
Passed	True
Test	Noise
Topology	DiamondMerge
Status	Found
Recall	2
5 total	
⌕ ⏴ rows 1-5 of 6 ⏵ ⌵	

▽

```
In[94]:= Dataset[{
  S14HistoryTest[s14, "TwoStageMerge"],
  S14BrokenTest[s14, "TwoStageMerge"],
  S14WrongTest[s14, "TwoStageMerge"],
  S14ConflictTest[s14, "TwoStageMerge"],
  S14NoiseTest[s14, "TwoStageMerge"],
  S14CompressionTest[s14, "TwoStageMerge"]
}]
```

Out[94]=

△

▽

Test	History
Topology	TwoStageMerge
RecallA	1
RecallB	3
5 total	
Test	Broken
Topology	TwoStageMerge
Status	Unknown
Passed	True
Test	WrongCue
Topology	TwoStageMerge
Status	Unknown
Passed	True
Test	Conflict
Topology	TwoStageMerge
Status	Ambiguous
Passed	True
Test	Noise
Topology	TwoStageMerge
Status	Found
Recall	2
5 total	
⏮ ⏪ rows 1–5 of 6 ⏩ ⏭	

```
In[95]:= r1 = S14Trials[s14, "DiamondMerge", 30, 20];  
r2 = S14Trials[s14, "TwoStageMerge", 30, 20];
```

```
Dataset[{  
  <|"Topology" → "DiamondMerge", "Passed" → Count[Lookup[r1, "Passed"], True],  
    "Cases" → Length[r1], "AllPassed" → And @@ Lookup[r1, "Passed"]|>,   
  <|"Topology" → "TwoStageMerge", "Passed" → Count[Lookup[r2, "Passed"], True],  
    "Cases" → Length[r2], "AllPassed" → And @@ Lookup[r2, "Passed"]|>  
}]
```

Out[97]=

Topology	Passed	Cases	AllPassed
DiamondMerge	30	30	True
TwoStageMerge	30	30	True

```

In[98]:= t1 = {
  S14HistoryTest[s14, "DiamondMerge"],
  S14BrokenTest[s14, "DiamondMerge"],
  S14WrongTest[s14, "DiamondMerge"],
  S14ConflictTest[s14, "DiamondMerge"],
  S14NoiseTest[s14, "DiamondMerge"],
  S14CompressionTest[s14, "DiamondMerge"]
};

t2 = {
  S14HistoryTest[s14, "TwoStageMerge"],
  S14BrokenTest[s14, "TwoStageMerge"],
  S14WrongTest[s14, "TwoStageMerge"],
  S14ConflictTest[s14, "TwoStageMerge"],
  S14NoiseTest[s14, "TwoStageMerge"],
  S14CompressionTest[s14, "TwoStageMerge"]
};

Dataset[{
  <|"Topology" → "DiamondMerge", "Passed" →
    Map[TrueQ[Lookup[#, "Passed", Lookup[#, "AllPassed", False]]] &, t1], "AllPassed" →
    And @@ Map[TrueQ[Lookup[#, "Passed", Lookup[#, "AllPassed", False]]] &, t1]>,
  <|"Topology" → "TwoStageMerge", "Passed" →
    Map[TrueQ[Lookup[#, "Passed", Lookup[#, "AllPassed", False]]] &, t2], "AllPassed" →
    And @@ Map[TrueQ[Lookup[#, "Passed", Lookup[#, "AllPassed", False]]] &, t2]>
}]

```

Out[100]=

Topology	Passed	AllPassed
DiamondMerge	True	True
	True	
	True	
	True	
	True	
	True	
TwoStageMerge	True	True
	True	
	True	
	True	
	True	
	True	

In[101]:=

```
ClearAll[B15Case, B15Raw, B15DecA, B15Prop2I, B15Prop4I,
  B15PropNamedI, B15BFS, B15Decision, B15PassQ, B15Fit, B15Base, B15Broken,
  B15Wrong, B15Conflict, B15HeldCases, B15Eval, B15Summary, B15Audit];
```

```
B15Raw[i_, h_] := AssociationThread[i["V"],
  Boole[Total@Lookup[h, Lookup[i["In"], #, {}], 0] > 0] & /@ i["V"]];
```

```
B15DecA[s_, i_, a_] := Catch[Module[{c = s["K"], vis = {}, nx, t, mn, z, n, k, v},
  Do[If[MemberQ[s["Values"], c], v = First@FirstPosition[s["Values"], c];
    Throw[<|"Status" -> "Found", "Value" -> v|>]];
  If[MemberQ[vis, c], Throw[<|"Status" -> "Loop", "Value" -> "Loop"|>]];
  AppendTo[vis, c];
  nx = Lookup[i["Out"], c, {}];
  If[nx === {}, Throw[<|"Status" -> "Unknown", "Value" -> "Unknown"|>]];
  If[Length[nx] === 1, n = First[nx], t = Lookup[a, nx, Infinity];
  mn = Min[t];
  If[mn === Infinity, Throw[<|"Status" -> "Unknown", "Value" -> "Unknown"|>]];
  z = Pick[nx, t, mn];
  If[Length[z] > 1, Throw[<|"Status" -> "Ambiguous", "Value" -> "Ambiguous"|>]];
  n = First[z]];
```

```
c = n, {k, 3000}]; <| "Status" → "Budget", "Value" → "Budget" |>]]];
```

```
B15Prop2I[i_, c_, w_, max_: 8] := Module[{v = i["V"], h, raw, h2, a = <| |>, new, k},
  h = AssociationThread[v, ConstantArray[0, Length[v]]];
  AssociateTo[h, c → 1];
  AssociateTo[a, c → 0];
  Do[raw = B15Raw[i, h];
    new = Select[v, Lookup[raw, #, 0] === 1 && ! KeyExistsQ[a, #] &];
    Scan[(AssociateTo[a, # → k]) &, new];
    h2 = AssociationThread[v,
      Boole[Lookup[raw, #, 0] === 1 && w.{1, Length@Lookup[i["In"], #, {}]} > 0] & /@ v];
    If[SameQ[h, h2], Break[]];
    h = h2, {k, max}]; a];
```

```
B15Prop4I[i_, c_, w_, max_: 8] := Module[{v = i["V"], h, raw, h2, a = <| |>, new, k, f},
  h = AssociationThread[v, ConstantArray[0, Length[v]]];
  AssociateTo[h, c → 1];
  AssociateTo[a, c → 0];
  Do[raw = B15Raw[i, h];
    new = Select[v, Lookup[raw, #, 0] === 1 && ! KeyExistsQ[a, #] &];
    Scan[(AssociateTo[a, # → k]) &, new];
    h2 = AssociationThread[v, Boole[Lookup[raw, #, 0] === 1 &&
      (f = {1, Length@Lookup[i["In"], #, {}], Length@Lookup[i["Out"], #, {}],
        Total[Length@Lookup[i["Out"], u, {}] & /@ Lookup[i["In"], #, {}]]};
      w.f > 0)] & /@ v];
    If[SameQ[h, h2], Break[]];
    h = h2, {k, max}]; a];
```

```
B15PropNamedI[i_, c_, w_, max_: 8] :=
  Module[{v = i["V"], h, sn, ag, raw, ns, na, h2, a = <| |>, new, k, ps, cnt, p, x},
    h = AssociationThread[v, ConstantArray[0, Length[v]]];
    sn = AssociationThread[v, ConstantArray[0, Length[v]]];
    ag = AssociationThread[v, ConstantArray[0, Length[v]]];
    AssociateTo[h, c → 1];
    AssociateTo[a, c → 0];
    Do[raw = B15Raw[i, h];
      ns = <| |>;
```

```

na = <| |>;
Do[If[Lookup[raw, x, 0] === 0, AssociateTo[ns, x → 0];
    AssociateTo[na, x → 0], ps = Select[Lookup[i["In"], x, {}], Lookup[h, #, 0] === 1 &];
    cnt = Length[ps];
    Which[cnt ≥ 2, AssociateTo[ns, x → 1];
        AssociateTo[na, x → 0], cnt === 1, p = First[ps];
        AssociateTo[ns, x → Lookup[sn, p, 0]];
        AssociateTo[na, x → If[Lookup[sn, p, 0] === 1, Min[3, Lookup[ag, p, 0] + 1], 0]],
        True, AssociateTo[ns, x → 0];
        AssociateTo[na, x → 0]], {x, v}];
new = Select[v, Lookup[raw, #, 0] === 1 && !KeyExistsQ[a, #] &];
Scan[(AssociateTo[a, # → k]) &, new];
h2 = AssociationThread[v,
    Boole[Lookup[raw, #, 0] === 1 && w.{1, Length@Lookup[i["In"], #, {}], Length@
        Lookup[i["Out"], #, {}], Lookup[ns, #, 0], Lookup[na, #, 0]} > 0] & /@ v];
If[SameQ[h, h2] && SameQ[sn, ns] && SameQ[ag, na], Break[]];
h = h2;
sn = AssociationThread[v, Lookup[ns, #, 0] & /@ v];
ag = AssociationThread[v, Lookup[na, #, 0] & /@ v, {k, max}]; a];

```

```

B15BFS[s_, q_] := Catch[Module[{g = s["Memory"], dg, c = s["K"], cue = s["Cues"][[q]],
    vis = {}, nx, d, mn, z, n, k, v}, dg = Graph[VertexList[g], S14F[g]];
Do[If[MemberQ[s["Values"], c], v = First@FirstPosition[s["Values"], c];
    Throw[<| "Status" → "Found", "Value" → v |>]];
If[MemberQ[vis, c], Throw[<| "Status" → "Loop", "Value" → "Loop" |>]];
AppendTo[vis, c];
nx = S14Next[g, c];
If[nx === {}, Throw[<| "Status" → "Unknown", "Value" → "Unknown" |>]];
If[Length[nx] === 1, n = First[nx], d = Quiet[GraphDistance[dg, cue, #] & /@ nx];
    d = d /. DirectedInfinity → Infinity;
    mn = Min[d];
    If[mn === Infinity, Throw[<| "Status" → "Unknown", "Value" → "Unknown" |>]];
    z = Pick[nx, d, mn];
    If[Length[z] > 1, Throw[<| "Status" → "Ambiguous", "Value" → "Ambiguous" |>]];
    n = First[z];
    c = n, {k, 3000}]; <| "Status" → "Budget", "Value" → "Budget" |>]];

```

```

B15Decision["Static2", p_, c_] :=
    B15DecA[c["State"], c["Index"], B15Prop2I[c["Index"], c["State"][[c["Cues"][[c["Cue"]]], p]]];

```

```

B15Decision["Static4", p_, c_] :=
  B15DecA[c["State"], c["Index"], B15Prop4I[c["Index"], c["State"]["Cues"][[c["Cue"]]], p]];
B15Decision["Named", p_, c_] := B15DecA[c["State"],
  c["Index"], B15PropNamedI[c["Index"], c["State"]["Cues"][[c["Cue"]]], p]];
B15Decision["Latent", p_, c_] := S14DecisionI[c["State"],
  c["Index"], p["TransitionParams"], p["GateWeights"], c["Cue"], 8];
B15Decision["BFS", _, c_] := B15BFS[c["State"], c["Cue"]];

B15PassQ[m_, p_, c_] := Module[{r = B15Decision[m, p, c]},
  If[c["ExpectedStatus"] === "Found", r["Status"] === "Found" &&
    r["Value"] === c["ExpectedValue"], r["Status"] === c["ExpectedStatus"]]];

B15Fit[m_] := Module[{c = S14TrainCases[], g, ord, alive, best, r},
  g = Switch[m, "Static2", Tuples[{Range[1, 3], Range[-2, 0]}], "Static4",
    Tuples[{Range[1, 4], Range[-1, 1], Range[-1, 1], Range[-2, 0]}], "Named",
    Tuples[{Range[1, 3], Range[-2, 0], Range[-1, 1], Range[-1, 1], Range[-1, 1]}]];
  ord = SortBy[c, Switch[#, "Type", "Broken", 1, "Conflict",
    2, "WrongCue", 3, "Positive", 4] &];
  alive = g;
  Do[alive = Select[alive, B15PassQ[m, #, x] &];
    If[alive === {}, Break[], {x, ord}];
  If[alive != {}, best = First@SortBy[alive, {Total@Abs[#] &, Count[#, Except[0]] &}];
  Return[<| "Model" → m, "Params" → best,
    "SearchSpace" → Length[g], "Score" → 42, "Accuracy" → 1., "Perfect" → True |>];
  r = <| "Params" → #, "Score" → Count[B15PassQ[m, #, x] & /@ c, True] |> & /@ g;
  best = First@MaximalBy[r, #["Score"] &]; <|
    "Model" → m, "Params" → best["Params"], "SearchSpace" → Length[g],
    "Score" → best["Score"], "Accuracy" → N[best["Score"]/42], "Perfect" → False |>];

B15Case[type_, s_, q_, st_, v_] := <|
  "Type" → type, "State" → s, "Index" → S14Index[s["Memory"]],
  "Cue" → q, "ExpectedStatus" → st, "ExpectedValue" → v |>;

B15Base[top_, d_, j_, q_, br_, seed_] := Module[{s, p, m, x}, s = S14Init[4, 8];
  SeedRandom[seed + 999];
  x = RandomSample[Range[8]];
  s = S14Permute[s, x];
  p = S14TestProgram[s, top, d, j, q, br, seed];
  m = S14Replay[s, p];
  m = S14Relabel[m, seed + 5000];
  {s, p, m}];

B15Broken[top_, d_, j_, q_, br_, seed_] := Module[{s, p, x, m, k}, s = S14Init[4, 8];

```

```

SeedRandom[seed + 999];
s = S14Permute[s, RandomSample[Range[8]]];
p = S14TestProgram[s, top, d, j, q, br, seed];
k = Max[1, Ceiling[d/2]];
x = First@FirstPosition[p["PathTriples"], p["CorrectPathTriples"][[k]];
m = S14Replay[s, Join[p, <|"PathTriples" → Delete[p["PathTriples"], x]|>]];
m = S14Relabel[m, seed + 5000];
B15Case["Broken", m, q, "Unknown", "Unknown"];

B15Wrong[top_, d_, j_, q_, br_, seed_] :=
Module[{x, m, w}, x = B15Base[top, d, j, q, br, seed];
m = x[[3]];
w = 1 + Mod[q, 8];
B15Case["WrongCue", m, w, "Unknown", "Unknown"];

B15Conflict[top_, d_, q_, br_, seed_] := Module[{s, a, b, m}, s = S14Init[4, 8];
SeedRandom[seed + 999];
s = S14Permute[s, RandomSample[Range[8]]];
a = S14TestProgram[s, top, d, 1, q, br, seed, 1];
b = S14TestProgram[s, top, d, 4, q, br, seed + 1, 2];
m = Fold[S14Replay, s, {a, b}];
m = S14Relabel[m, seed + 5000];
B15Case["Conflict", m, q, "Ambiguous", "Ambiguous"];

B15HeldCases[top_] :=
Module[{p, b, w, c, x, d, j, q, seed}, p = Flatten[Table[q = 1 + Mod[d + j - 2, 8];
seed = 600 000 + 100 * d + j;
x = B15Base[top, d, j, q, 3, seed];
B15Case["Positive", x[[3]], q, "Found", j], {d, 5, 12}, {j, 1, 4}], 1];
b = Table[j = 1 + Mod[d - 5, 4];
q = 1 + Mod[2 * d - 1, 8];
B15Broken[top, d, j, q, 3, 700 000 + d], {d, 5, 12}];
w = Table[j = 1 + Mod[d - 4, 4];
q = 1 + Mod[3 * d - 1, 8];
B15Wrong[top, d, j, q, 3, 800 000 + d], {d, 5, 12}];
c = Table[q = 1 + Mod[d, 8];
B15Conflict[top, d, q, 3, 900 000 + d], {d, {5, 7, 9, 12}}];
Join[p, b, w, c];

B15Eval[m_, p_, c_, name_] :=
Module[{t, r, types, pt}, t = AbsoluteTiming[r = B15PassQ[m, p, #] & /@ c;][[1]];
types = DeleteDuplicates@Lookup[c, "Type"];
pt = Association@Table[x → Count[Pick[r, Lookup[c, "Type"], x], True], {x, types}]; <|

```

```

"Model" → m, "Set" → name, "Cases" → Length[c],
"Passed" → Count[r, True], "Accuracy" → N@Mean[Boole /@ r],
"PassedByType" → pt, "TimeSec" → t, "AllPassed" → And @@ r |>];

B15Audit[] := <|"TrainCases" → Length@S14TrainCases[],
  "TrainingTopologies" → {"Chain", "Parallel2", "InternalMerge"},
  "HeldOutTopologies" → {"DiamondMerge", "TwoStageMerge"}, "StaticCandidateLists" →
    False, "NamedCandidateLists" → False, "LatentCandidateLists" → False,
  "BFSUsesGlobalGraphSearch" → True, "LatentRetrained" → False,
  "AllPassed" → And[Length@S14TrainCases[] === 42, TrueQ[s14["Perfect"]]] |>;
ClearAll[B15PassQ, B15Fit];

B15PassQ[m_, p_, c_Association] := Module[{r},
  r = B15Decision[m, p, c];
  If[c["ExpectedStatus"] === "Found",
    r["Status"] === "Found" && r["Value"] === c["ExpectedValue"],
    r["Status"] === c["ExpectedStatus"]]
];

B15Fit[m_String] := Module[{c, g, ord, alive, best, r},
  c = S14TrainCases[];
  g = Switch[m,
    "Static2", Tuples[{Range[1, 3], Range[-2, 0]}],
    "Static4", Tuples[{Range[1, 4], Range[-1, 1], Range[-1, 1], Range[-2, 0]}],
    "Named", Tuples[{Range[1, 3], Range[-2, 0], Range[-1, 1], Range[-1, 1], Range[-1, 1]}]
];
  ord = SortBy[c, Switch[#, "Type", "Broken",
    1, "Conflict", 2, "WrongCue", 3, "Positive", 4, _, 5] &];
  alive = g;
  Do[
    alive = Select[alive, Function[p, B15PassQ[m, p, cc]]];
    If[alive === {}, Break[]],
    {cc, ord}
];
  If[alive != {},
    best = First@SortBy[alive, {Total@Abs[#] &, Count[#], Except[0]} &];
    Return[<|"Model" → m, "Params" → best, "SearchSpace" → Length[g],
      "Score" → Length[c], "Accuracy" → 1., "Perfect" → True |>]
];
  r = Map[
    Function[p,
      <|"Params" → p, "Score" → Count[Map[Function[cc, B15PassQ[m, p, cc]], c], True] |>
    ],

```

```

g
];
best = First@MaximalBy[r, #["Score"] &];
<|"Model" → m, "Params" → best["Params"], "SearchSpace" → Length[g],
  "Score" → best["Score"], "Accuracy" → N[best["Score"]/Length[c]], "Perfect" → False |>
];

```

In[126]:=

```
Dataset[{B15Audit[]}]
```

Out[126]=

TrainCases	TrainingTopologies	HeldOutTopologies	StaticCandidateLists	Named
42	{Chain, Parallel2, InternalMerge}	{DiamondMerge, TwoStageMerge}	False	False

In[127]:=

```
Clear[b2, b4, bn];
```

```

b2 = B15Fit["Static2"];
b4 = B15Fit["Static4"];
bn = B15Fit["Named"];

```

```
Dataset[{b2, b4, bn}]
```

Out[131]=

Model	Params	SearchSpace	Score	Accuracy	Perfect
Static2	{3, -1}	9	40	0.952381	False
Static4	{2, -1, 1, -2}	108	40	0.952381	False
Named	{2, -1, 0, 1, -1}	243	42	1.0	True

In[132]:=

```

hd = B15HeldCases["DiamondMerge"];
ht = B15HeldCases["TwoStageMerge"];

```

```

Length[hd]
Length[ht]
Counts[Lookup[hd, "Type"]]

```

Out[134]=

52

Out[135]=

52

Out[136]=

```
<|Positive → 32, Broken → 8, WrongCue → 8, Conflict → 4|>
```

In[137]:=

```
ClearAll[B15DecA, B15BFS, B15Decision, B15Eval];
```

```

B15DecA[s_Association, i_Association, a_Association] :=
  Catch[Module[{c = s["K"], vis = {}, nx, t, mn, z, n, k, vm},
    vm = AssociationThread[s["Values"], Range[Length[s["Values"]]]];
    Do[
      If[KeyExistsQ[vm, c], Throw[<|"Status" → "Found", "Value" → vm[c]|>]];
      If[MemberQ[vis, c], Throw[<|"Status" → "Loop", "Value" → "Loop"|>]];
      AppendTo[vis, c];
      nx = Lookup[i["Out"], c, {}];
      If[nx === {}, Throw[<|"Status" → "Unknown", "Value" → "Unknown"|>]];
      If[
        Length[nx] === 1,
        n = nx[[1]],
        t = Lookup[a, nx, Infinity];
        mn = Min[t];
        If[mn === Infinity, Throw[<|"Status" → "Unknown", "Value" → "Unknown"|>]];
        z = Pick[nx, t, mn];
        If[Length[z] != 1, Throw[<|"Status" → If[z === {}, "Unknown", "Ambiguous"],
          "Value" → If[z === {}, "Unknown", "Ambiguous"]|>]];
        n = z[[1]]
      ];
      c = n,
      {k, 1, 3000}
    ];
    <|"Status" → "Budget", "Value" → "Budget"|>
  ]];

```

```

B15BFS[s_Association, q_Integer] :=
  Catch[Module[{g = s["Memory"], dg, i, c = s["K"], cue, vis = {}, nx, d, mn, z, n, k, vm},
    i = S14Index[g];
    dg = Graph[VertexList[g], S14F[g]];
    cue = s["Cues"][[q]];
    vm = AssociationThread[s["Values"], Range[Length[s["Values"]]]];
    Do[
      If[KeyExistsQ[vm, c], Throw[<|"Status" → "Found", "Value" → vm[c]|>]];
      If[MemberQ[vis, c], Throw[<|"Status" → "Loop", "Value" → "Loop"|>]];
      AppendTo[vis, c];
      nx = Lookup[i["Out"], c, {}];
      If[nx === {}, Throw[<|"Status" → "Unknown", "Value" → "Unknown"|>]];
      If[

```

```

Length[nx] === 1,
n = nx[[1]],
d = Quiet[GraphDistance[dg, cue, #] & /@ nx];
d = d /. DirectedInfinity → Infinity;
mn = Min[d];
If[mn === Infinity, Throw[<|"Status" → "Unknown", "Value" → "Unknown" |>]];
z = Pick[nx, d, mn];
If[Length[z] != 1, Throw[<|"Status" → If[z === {}, "Unknown", "Ambiguous"],
    "Value" → If[z === {}, "Unknown", "Ambiguous"] |>]];
n = z[[1]]
];
c = n,
{k, 1, 3000}
];
<|"Status" → "Budget", "Value" → "Budget" |>
]];

```

```

B15Decision["Static2", p_, c_Association] :=
    B15DecA[c["State"], c["Index"], B15Prop2I[c["Index"], c["State"]["Cues"][[c["Cue"]]], p]];
B15Decision["Static4", p_, c_Association] :=
    B15DecA[c["State"], c["Index"], B15Prop4I[c["Index"], c["State"]["Cues"][[c["Cue"]]], p]];
B15Decision["Named", p_, c_Association] := B15DecA[c["State"],
    c["Index"], B15PropNamedI[c["Index"], c["State"]["Cues"][[c["Cue"]]], p]];
B15Decision["Latent", p_Association, c_Association] := Module[{r},
    r = S14PropI[c["Index"],
        c["State"]["Cues"][[c["Cue"]]], p["TransitionParams"], p["GateWeights"], 8];
    B15DecA[c["State"], c["Index"], r["Arrival"]]
];
B15Decision["BFS", _, c_Association] := B15BFS[c["State"], c["Cue"]];

```

```

B15Eval[m_, p_, c_List, name_] := Module[{r, t, types, pt},
    {t, r} = AbsoluteTiming[Map[Function[x, B15PassQ[m, p, x]], c]];
    types = DeleteDuplicates[Lookup[c, "Type"]];
    pt = Association@Table[x → Count[Pick[r, Lookup[c, "Type"], x], True], {x, types}];
    <|"Model" → m, "Set" → name, "Cases" → Length[c], "Passed" → Count[r, True], "Accuracy" →
        N@Mean[Boole /@ r], "PassedByType" → pt, "TimeSec" → t, "AllPassed" → And @@ r |>
];

```

In[146]:=

```

x1 = B15Eval["Static2", b2["Params"], hd, "DiamondMerge"];
Dataset[{x1}]

```

Out[147]=

Model	Set	Cases	Passed	Accuracy	PassedByType	
Static2	DiamondMerge	52	44	0.846154	Positive	3
					Broken	0
					WrongCue	8
					Conflict	4

In[148]:=

```
x2 = B15Eval["Static4", b4["Params"], hd, "DiamondMerge"];
Dataset[{x2}]
```

Out[149]=

Model	Set	Cases	Passed	Accuracy	PassedByType	
Static4	DiamondMerge	52	44	0.846154	Positive	3
					Broken	0
					WrongCue	8
					Conflict	4

In[150]:=

```
x3 = B15Eval["Named", bn["Params"], hd, "DiamondMerge"];
Dataset[{x3}]
```

Out[151]=

Model	Set	Cases	Passed	Accuracy	PassedByType	
Named	DiamondMerge	52	52	1.0	Positive	32
					Broken	8
					WrongCue	8
					Conflict	4

In[152]:=

```
x4 = B15Eval["Latent", s14, hd, "DiamondMerge"];
Dataset[{x4}]
```

Out[153]=

Model	Set	Cases	Passed	Accuracy	PassedByType	
Latent	DiamondMerge	52	52	1.0	Positive	32
					Broken	8
					WrongCue	8
					Conflict	4

In[154]:=

```
x5 = B15Eval["BFS", None, hd, "DiamondMerge"];
Dataset[{x5}]
```

Out[155]=

Model	Set	Cases	Passed	Accuracy	PassedByType		Tir
BFS	DiamondMerge	52	12	0.230769	Positive	0	0.0
					Broken	8	
					WrongCue	4	
					Conflict	0	

In[158]:=

```
x6 = B15Eval["Static2", b2["Params"], ht, "TwoStageMerge"];
x7 = B15Eval["Static4", b4["Params"], ht, "TwoStageMerge"];
x8 = B15Eval["Named", bn["Params"], ht, "TwoStageMerge"];
x9 = B15Eval["Latent", s14, ht, "TwoStageMerge"];
x10 = B15Eval["BFS", None, ht, "TwoStageMerge"];
```

In[163]:=

```
res = {x1, x2, x3, x4, x5, x6, x7, x8, x9, x10};
```

```
Dataset[
  KeyTake[#,
    {"Model", "Set", "Passed", "Cases",
      "Accuracy", "PassedByType", "TimeSec", "AllPassed"}
  ] & /@ res
]
```

Out[164]=

Model	Set	Passed	Cases	Accuracy	PassedByType		
					Positive	Broken	Wrong
Static2	DiamondMerge	44	52	0.846154	32	0	8
Static4	DiamondMerge	44	52	0.846154	32	0	8
Named	DiamondMerge	52	52	1.0	32	8	8
Latent	DiamondMerge	52	52	1.0	32	8	8
BFS	DiamondMerge	12	52	0.230769	0	8	4
Static2	TwoStageMerge	44	52	0.846154	32	0	8
Static4	TwoStageMerge	44	52	0.846154	32	0	8
Named	TwoStageMerge	52	52	1.0	32	8	8
Latent	TwoStageMerge	52	52	1.0	32	8	8
BFS	TwoStageMerge	12	52	0.230769	0	8	4

In[165]:=

```
ClearAll[B16Reach, B16GlobalDecomp, B16Audit, B16Meta, B16Aggregate];
```

```
B16Reach[out_Association, start_] :=
  Module[{d = <|start → 0|>, f = {start}, nf = {}, u, v, k = 0},
    While[f != {},
      nf = {};
      Do[
        Do[
          If[! KeyExistsQ[d, v], AssociateTo[d, v → k + 1]; AppendTo[nf, v]],
          {v, Lookup[out, u, {}]}],
        {u, f}];
      f = DeleteDuplicates[nf];
      k++
```

```
];
<|"Dist" → d, "Visits" → Length[d]|>
];
```

```
B16GlobalDecomp[s_Association, q_Integer] := Catch[Module[
{i, ex, ev, rr, c = s["K"], cue, vis = {}, nx, good, k, vm},
i = S14Index[s["Memory"]];
ex = Keys[B16Reach[i["Out"], s["K"]]["Dist"]];
ev = AssociationThread[
i["V"],
If[MemberQ[ex, #], {}, Lookup[i["Out"], #, {}]] & /@ i["V"]
];
cue = s["Cues"][[q]];
rr = B16Reach[ev, cue];
vm = AssociationThread[s["Values"], Range[Length[s["Values"]]]];
Do[
If[KeyExistsQ[vm, c],
Throw[<|"Status" → "Found", "Value" → vm[c],
"GlobalVisits" → rr["Visits"], "ExecNodes" → Length[ex]|>]
];
If[MemberQ[vis, c],
Throw[<|"Status" → "Loop", "Value" → "Loop",
"GlobalVisits" → rr["Visits"], "ExecNodes" → Length[ex]|>]
];
AppendTo[vis, c];
nx = Lookup[i["Out"], c, {}];
If[nx === {},
Throw[<|"Status" → "Unknown", "Value" → "Unknown",
"GlobalVisits" → rr["Visits"], "ExecNodes" → Length[ex]|>]
];
If[
Length[nx] === 1,
c = First[nx],
good = Select[nx, KeyExistsQ[rr["Dist"], #] &];
Which[
good === {},
Throw[<|"Status" → "Unknown", "Value" → "Unknown",
"GlobalVisits" → rr["Visits"], "ExecNodes" → Length[ex]|>],
Length[good] > 1,
Throw[<|"Status" → "Ambiguous", "Value" → "Ambiguous",
```

```

      "GlobalVisits" → rr["Visits"], "ExecNodes" → Length[ex] |>,
True,
c = First[good]
],
],
{k, 1, 3000}
];
<| "Status" → "Budget", "Value" → "Budget",
    "GlobalVisits" → rr["Visits"], "ExecNodes" → Length[ex] |>
]];

```

```

B15Decision["NaiveSP", _, c_Association] := B15BFS[c["State"], c["Cue"]];
B15Decision["GlobalDecomp", _, c_Association] := B16GlobalDecomp[c["State"], c["Cue"]];

```

```

B16Audit[] := Module[{d},
d = {DownValues[B16Reach], DownValues[B16GlobalDecomp]};
<|
"UsesGeneratorGroundTruth" → False,
"ExplicitCandidateList" → False,
"UsesGlobalGraphTraversal" → True,
"UsesGraphDistance" → False,
"UsesKReachabilityDecomposition" → True,
"RetrainingRequired" → False,
"AllPassed" → And[
FreeQ[d, "Candidates"],
FreeQ[d, S14TestProgram | S14TrainProgram | CorrectPathTriples],
FreeQ[d, GraphDistance]
]
|>
];

```

```

B16Meta[] := {
<| "Model" → "Static2", "Local" → True, "Stateful" → False,
    "GlobalSearch" → False, "HandNamedState" → False, "LearnsParams" → True |>,
<| "Model" → "Static4", "Local" → True, "Stateful" → False,
    "GlobalSearch" → False, "HandNamedState" → False, "LearnsParams" → True |>,
<| "Model" → "Named", "Local" → True, "Stateful" → True,
    "GlobalSearch" → False, "HandNamedState" → True, "LearnsParams" → True |>,
<| "Model" → "Latent", "Local" → True, "Stateful" → True,
    "GlobalSearch" → False, "HandNamedState" → False, "LearnsParams" → True |>,
<| "Model" → "NaiveSP", "Local" → False, "Stateful" → False,
    "GlobalSearch" → True, "HandNamedState" → False, "LearnsParams" → False |>,

```

```
<|"Model" → "GlobalDecomp", "Local" → False, "Stateful" → False,
  "GlobalSearch" → True, "HandNamedState" → False, "LearnsParams" → False |>
};
```

```
B16Aggregate[r_List] := Module[{ms, rr},
ms = DeleteDuplicates[Lookup[r, "Model"]];
Table[
rr = Select[r, Lookup[#, "Model"] === m &];
<|
"Model" → m,
"Passed" → Total[Lookup[rr, "Passed"]],
"Cases" → Total[Lookup[rr, "Cases"]],
"Accuracy" → N[Total[Lookup[rr, "Passed"]]/Total[Lookup[rr, "Cases"]]],
"PassedByType" → Merge[Lookup[rr, "PassedByType"], Total],
"TimeSec" → Total[Lookup[rr, "TimeSec"]],
"AllPassed" → And @@ Lookup[rr, "AllPassed"]
|>,
{m, ms}]
];
```

In[173]:=

```
Dataset[{B16Audit[]}]
Dataset[B16Meta[]]
```

Out[173]=

UsesGeneratorGroundTruth	False
ExplicitCandidateList	False
UsesGlobalGraphTraversal	True
UsesGraphDistance	False
UsesKReachabilityDecomposition	True
RetrainingRequired	False
AllPassed	True

Out[174]=

Model	Local	Stateful	GlobalSearch	HandNamedState	LearnsParams
Static2	True	False	False	False	True
Static4	True	False	False	False	True
Named	True	True	False	True	True
Latent	True	True	False	False	True
NaiveSP	False	False	True	False	False
GlobalDecomp	False	False	True	False	False

In[175]:=

```

res2 = {
  B15Eval["Static2", b2["Params"], hd, "DiamondMerge"],
  B15Eval["Static4", b4["Params"], hd, "DiamondMerge"],
  B15Eval["Named", bn["Params"], hd, "DiamondMerge"],
  B15Eval["Latent", s14, hd, "DiamondMerge"],
  B15Eval["NaiveSP", None, hd, "DiamondMerge"],
  B15Eval["GlobalDecomp", None, hd, "DiamondMerge"],

  B15Eval["Static2", b2["Params"], ht, "TwoStageMerge"],
  B15Eval["Static4", b4["Params"], ht, "TwoStageMerge"],
  B15Eval["Named", bn["Params"], ht, "TwoStageMerge"],
  B15Eval["Latent", s14, ht, "TwoStageMerge"],
  B15Eval["NaiveSP", None, ht, "TwoStageMerge"],
  B15Eval["GlobalDecomp", None, ht, "TwoStageMerge"]
};

```

In[176]:=

```

Dataset[
  KeyTake[#,
    {"Model", "Set", "Passed", "Cases",
      "Accuracy", "PassedByType", "TimeSec", "AllPassed"}
  ] & /@ res2
]

```

Out[176]=

Model	Set	Passed	Cases	Accuracy		
					Positive	B
Static2	DiamondMerge	44	52	0.846154	32	0
Static4	DiamondMerge	44	52	0.846154	32	0
Named	DiamondMerge	52	52	1.0	32	8
Latent	DiamondMerge	52	52	1.0	32	8
NaiveSP	DiamondMerge	12	52	0.230769	0	8
GlobalDecomp	DiamondMerge	52	52	1.0	32	8
Static2	TwoStageMerge	44	52	0.846154	32	0
Static4	TwoStageMerge	44	52	0.846154	32	0
Named	TwoStageMerge	52	52	1.0	32	8
Latent	TwoStageMerge	52	52	1.0	32	8
NaiveSP	TwoStageMerge	12	52	0.230769	0	8
GlobalDecomp	TwoStageMerge	52	52	1.0	32	8

In[177]:=

Dataset[B16Aggregate[res2]]

Out[177]=

Model	Passed	Cases	Accuracy	PassedByType				Tir
				Positive	Broken	WrongCue	Conflict	
Static2	88	104	0.846154	64	0	16	8	2.2
Static4	88	104	0.846154	64	0	16	8	2.5
Named	104	104	1.0	64	16	16	8	3.4
Latent	104	104	1.0	64	16	16	8	2.9
NaiveSP	24	104	0.230769	0	16	8	0	0.1
GlobalDecomp	104	104	1.0	64	16	16	8	0.3

In[178]:=

ClearAll[B17GraphAudit, B17PosCase, B17DepthCurve, B17Trace, B17Summary];

```

B17GraphAudit[] := Module[{g},
g = Graph[{DirectedEdge[1, 2], DirectedEdge[2, 3]}];
<|"ForwardDistance" → GraphDistance[g, 1, 3],
  "ReverseDistance" → GraphDistance[g, 3, 1], "DirectionCorrect" →
    And[GraphDistance[g, 1, 3] === 2, GraphDistance[g, 3, 1] === Infinity]|>
];

```

```

B17PosCase[top_, d_, j_, seed_] := Module[{q, x},
q = 1 + Mod[d + j - 2, 8];
x = B15Base[top, d, j, q, 3, seed];
B15Case["Positive", x[[3]], q, "Found", j]
];

```

```

B17DepthCurve[top_] := Table[
Module[{c, rn, rg},
c = Table[B17PosCase[top, d, j, 1700000 + 100*d + j], {j, 1, 4}];
rn = B15PassQ["NaiveSP", None, #] & /@ c;
rg = B15PassQ["GlobalDecomp", None, #] & /@ c;
<|"Topology" → top, "Depth" → d,
  "NaivePassed" → Count[rn, True], "GlobalPassed" → Count[rg, True], "Cases" → 4|>
],

```

```
{d, 1, 12}
];
```

```
B17Trace[c_Association] := Module[{s = c["State"],
  i = c["Index"], g, dg, cue, cur, vis = {}, nx, dist, mn, best, rows = {}, k},
  g = s["Memory"];
  dg = Graph[VertexList[g], S14F[g]];
  cue = s["Cues"][[c["Cue"]]];
  cur = s["K"];
  Do[
  If[MemberQ[s["Values"], cur], Break[]];
  If[MemberQ[vis, cur],
    AppendTo[rows, <|"Step" → k, "Node" → cur, "Status" → "Loop"|>];
    Break[]];
  AppendTo[vis, cur];
  nx = Lookup[i["Out"], cur, {}];
  If[nx === {}, AppendTo[rows, <|"Step" → k, "Node" → cur, "Status" → "Unknown"|>];
  Break[]];
  dist = Quiet[GraphDistance[dg, cue, #] & /@ nx] /. DirectedInfinity → Infinity;
  mn = Min[dist];
  best = If[mn === Infinity, {}, Pick[nx, dist, mn]];
  AppendTo[
    rows, <|"Step" → k, "Node" → cur, "Branches" → Length[nx], "Distances" → dist,
      "Finite" → Count[dist, _?NumericQ], "BestCount" → Length[best], "Status" → Which[
        mn === Infinity, "Unknown", Length[best] > 1, "Ambiguous", True, "Continue"]|>];
  If[mn === Infinity || Length[best] != 1, Break[]];
  cur = First[best],
  {k, 1, 30}
  ];
  Dataset[rows]
  ];
```

```
B17Summary[] := Module[{pd, pt, nd, nt, gd, gt},
  pd = Select[hd, #["Type"] === "Positive" &];
  pt = Select[ht, #["Type"] === "Positive" &];
  nd = B15PassQ["NaiveSP", None, #] & /@ pd;
  nt = B15PassQ["NaiveSP", None, #] & /@ pt;
  gd = B15PassQ["GlobalDecomp", None, #] & /@ pd;
  gt = B15PassQ["GlobalDecomp", None, #] & /@ pt;
```

```
{
  <|"Model" → "NaiveSP", "Topology" → "DiamondMerge",
    "Passed" → Count[nd, True], "Cases" → Length[nd]|>,
  <|"Model" → "NaiveSP", "Topology" → "TwoStageMerge",
    "Passed" → Count[nt, True], "Cases" → Length[nt]|>,
  <|"Model" → "GlobalDecomp", "Topology" → "DiamondMerge",
    "Passed" → Count[gd, True], "Cases" → Length[gd]|>,
  <|"Model" → "GlobalDecomp", "Topology" → "TwoStageMerge",
    "Passed" → Count[gt, True], "Cases" → Length[gt]|>
}
];
```

In[184]:=

```
Dataset[{B17GraphAudit[]}]
```

Out[184]=

ForwardDistance	ReverseDistance	DirectionCorrect
2	∞	True

In[185]:=

```
ForwardDistance = 2
ReverseDistance = Infinity
DirectionCorrect = True
```

Out[185]=

2

Out[186]=

 ∞

Out[187]=

True

In[188]:=

```
Dataset[B17DepthCurve["DiamondMerge"]]  
Dataset[B17DepthCurve["TwoStageMerge"]]
```

Out[188]=

Topology	Depth	NaivePassed	GlobalPassed	Cases
DiamondMerge	1	0	4	4
DiamondMerge	2	0	4	4
DiamondMerge	3	0	4	4
DiamondMerge	4	0	4	4
DiamondMerge	5	0	4	4
DiamondMerge	6	0	4	4
DiamondMerge	7	0	4	4
DiamondMerge	8	0	4	4
DiamondMerge	9	0	4	4
DiamondMerge	10	0	4	4
DiamondMerge	11	0	4	4
DiamondMerge	12	0	4	4

Out[189]=

Topology	Depth	NaivePassed	GlobalPassed	Cases
TwoStageMerge	1	0	4	4
TwoStageMerge	2	0	4	4
TwoStageMerge	3	0	4	4
TwoStageMerge	4	0	4	4
TwoStageMerge	5	0	4	4
TwoStageMerge	6	0	4	4
TwoStageMerge	7	0	4	4
TwoStageMerge	8	0	4	4
TwoStageMerge	9	0	4	4
TwoStageMerge	10	0	4	4
TwoStageMerge	11	0	4	4
TwoStageMerge	12	0	4	4

In[190]:=

Dataset[B17Summary[]]

Out[190]=

Model	Topology	Passed	Cases
NaiveSP	DiamondMerge	0	32
NaiveSP	TwoStageMerge	0	32
GlobalDecomp	DiamondMerge	32	32
GlobalDecomp	TwoStageMerge	32	32

In[191]:=

```
fp = SelectFirst[hd, #["Type"] === "Positive" && ! B15PassQ["NaiveSP", None, #] &];  
B17Trace[fp]
```

Out[192]=

Step	Node	Branches	Distances			Finite	BestCount	Status
1	2101	3	4	$\infty[1]$	$\infty[1]$	1	0	Continue

In[193]:=

```
ft = SelectFirst[ht, #["Type"] === "Positive" && ! B15PassQ["NaiveSP", None, #] &];  
B17Trace[ft]
```

Out[194]=

Step	Node	Branches	Distances			Finite	BestCount	Status
1	2334	3	6	$\infty[1]$	$\infty[1]$	1	0	Continue

In[195]:=

ClearAll[B15BFS, B17Trace];**B15BFS[s_Association, q_Integer] :=**

```
  Catch[Module[{g = s["Memory"], dg, i, c = s["K"], cue, vis = {}, nx, d, mn, z, n, k, vm},
```

```
    i = S14Index[g];
```

```
    dg = Graph[VertexList[g], S14F[g]];
```

```
    cue = s["Cues"][[q]];
```

```
    vm = AssociationThread[s["Values"], Range[Length[s["Values"]]]];
```

```
    Do[
```

```
      If[KeyExistsQ[vm, c], Throw[<| "Status" → "Found", "Value" → vm[c] |>]];
```

```
      If[MemberQ[vis, c], Throw[<| "Status" → "Loop", "Value" → "Loop" |>]];
```

```
      AppendTo[vis, c];
```

```
      nx = Lookup[i["Out"], c, {}];
```

```
      If[nx === {}, Throw[<| "Status" → "Unknown", "Value" → "Unknown" |>]];
```

```
      If[
```

```
        Length[nx] === 1,
```

```
        n = First[nx],
```

```
        d = Quiet[GraphDistance[dg, cue, #] & /@ nx] /. DirectedInfinity[] → Infinity;
```

```

mn = Min[d];
If[mn === Infinity, Throw[<|"Status" → "Unknown", "Value" → "Unknown" |>]];
z = Pick[nx, d, mn];
If[Length[z] != 1, Throw[<|"Status" → If[z === {}, "Unknown", "Ambiguous"],
    "Value" → If[z === {}, "Unknown", "Ambiguous"] |>]];
n = First[z];
];
c = n,
{k, 1, 3000}
];
<|"Status" → "Budget", "Value" → "Budget" |>
]];

```

```

B17Trace[c_Association] := Module[{s = c["State"],
    i = c["Index"], g, dg, cue, cur, vis = {}, nx, dist, mn, best, rows = {}, k},
g = s["Memory"];
dg = Graph[VertexList[g], S14F[g]];
cue = s["Cues"][[c["Cue"]]];
cur = s["K"];
Do[
If[MemberQ[s["Values"], cur], Break[]];
If[MemberQ[vis, cur],
    AppendTo[rows, <|"Step" → k, "Node" → cur, "Status" → "Loop" |>];
    Break[]];
AppendTo[vis, cur];
nx = Lookup[i["Out"], cur, {}];
If[nx === {}, AppendTo[rows, <|"Step" → k, "Node" → cur, "Status" → "Unknown" |>];
    Break[]];
dist = Quiet[GraphDistance[dg, cue, #] & /@ nx] /. DirectedInfinity[_] → Infinity;
mn = Min[dist];
best = If[mn === Infinity, {}, Pick[nx, dist, mn]];
AppendTo[
    rows, <|"Step" → k, "Node" → cur, "Branches" → Length[nx], "Distances" → dist,
        "Finite" → Count[dist, _?NumericQ], "BestCount" → Length[best], "Status" → Which[
            mn === Infinity, "Unknown", Length[best] > 1, "Ambiguous", True, "Continue"] |>];
If[mn === Infinity || Length[best] != 1, Break[]];
cur = First[best],
{k, 1, 30}
];
Dataset[rows]
];

```

In[198]:=

```
Dataset[B17DepthCurve["DiamondMerge"]]
Dataset[B17DepthCurve["TwoStageMerge"]]
Dataset[B17Summary[]]
```

Out[198]=

Topology	Depth	NaivePassed	GlobalPassed	Cases
DiamondMerge	1	4	4	4
DiamondMerge	2	4	4	4
DiamondMerge	3	4	4	4
DiamondMerge	4	4	4	4
DiamondMerge	5	4	4	4
DiamondMerge	6	4	4	4
DiamondMerge	7	4	4	4
DiamondMerge	8	4	4	4
DiamondMerge	9	4	4	4
DiamondMerge	10	4	4	4
DiamondMerge	11	4	4	4
DiamondMerge	12	4	4	4

Out[199]=

Topology	Depth	NaivePassed	GlobalPassed	Cases
TwoStageMerge	1	4	4	4
TwoStageMerge	2	4	4	4
TwoStageMerge	3	4	4	4
TwoStageMerge	4	4	4	4
TwoStageMerge	5	4	4	4
TwoStageMerge	6	4	4	4
TwoStageMerge	7	4	4	4
TwoStageMerge	8	4	4	4
TwoStageMerge	9	4	4	4
TwoStageMerge	10	4	4	4
TwoStageMerge	11	4	4	4
TwoStageMerge	12	4	4	4

Out[200]=

Model	Topology	Passed	Cases
NaiveSP	DiamondMerge	32	32
NaiveSP	TwoStageMerge	32	32
GlobalDecomp	DiamondMerge	32	32
GlobalDecomp	TwoStageMerge	32	32

In[201]:=

```
fp = SelectFirst[hd, #["Type"] === "Positive" && !B15PassQ["NaiveSP", None, #] &];
If[AssociationQ[fp], B17Trace[fp], "NoFailedPositive"]
```

Out[202]=

NoFailedPositive

In[203]:=

```
res3 = {
  B15Eval["Static2", b2["Params"], hd, "DiamondMerge"],
  B15Eval["Static4", b4["Params"], hd, "DiamondMerge"],
  B15Eval["Named", bn["Params"], hd, "DiamondMerge"],
  B15Eval["Latent", s14, hd, "DiamondMerge"],
  B15Eval["NaiveSP", None, hd, "DiamondMerge"],
  B15Eval["GlobalDecomp", None, hd, "DiamondMerge"],
  B15Eval["Static2", b2["Params"], ht, "TwoStageMerge"],
  B15Eval["Static4", b4["Params"], ht, "TwoStageMerge"],
  B15Eval["Named", bn["Params"], ht, "TwoStageMerge"],
  B15Eval["Latent", s14, ht, "TwoStageMerge"],
  B15Eval["NaiveSP", None, ht, "TwoStageMerge"],
  B15Eval["GlobalDecomp", None, ht, "TwoStageMerge"]
};
```

Dataset[B16Aggregate[res3]]

Out[204]=

Model	Passed	Cases	Accuracy	PassedByType				Tir
				Positive	Broken	WrongCue	Conflict	
Static2	88	104	0.846154	64	0	16	8	2.2
Static4	88	104	0.846154	64	0	16	8	2.5
Named	104	104	1.0	64	16	16	8	3.3
Latent	104	104	1.0	64	16	16	8	2.9
NaiveSP	88	104	0.846154	64	0	16	8	0.1
GlobalDecomp	104	104	1.0	64	16	16	8	0.3

In[205]:=

```
Dataset@Select[
  res3,
  #["Model"] === "NaiveSP" &
]
```

Out[205]=

Model	Set	Cases	Passed	Accuracy	PassedByType
NaiveSP	DiamondMerge	52	44	0.846154	Positive
					Broken
					WrongCue
					Conflict
NaiveSP	TwoStageMerge	52	44	0.846154	Positive
					Broken
					WrongCue
					Conflict